

大模型辅助应用程序审计方法综述

A Survey of Large Language Model Assisted Application Auditing

匿名作者

代码审计与可信执行环境研究方向

Abstract

大语言模型 (Large Language Models, LLMs) 已从通用语言生成工具演化为程序分析与安全审计中的语义组件。本文基于 200 余篇文献和 158 条精选引用, 构建面向应用程序审计的结构化综述框架, 将现有方法归纳为十类: 直接 LLM 代码审查、漏洞检测与修复、LLM 与静态分析协同、检索增强与结构化表示增强、智能体/执行链审计、prompt 与安全感知微调、benchmark 与评估协议、模型完整性与后门防御、LLM 增强安全测试、以及 LLM 代码生成安全。

本文通过 5 张结构图和 10 张比较表系统呈现: (1) 方法分类体系图展示十类方法的层次关系; (2) 技术演进时间线标注 2013-2026 关键里程碑; (3) 多阶段审计流水线给出推荐架构与量化目标; (4) Agent 攻防全景图映射 5 类攻击与 5 类防御; (5) Benchmark 覆盖热力图揭示模型后门维度的评测空白。

关键发现包括: (1) LLM 审查接受率仅 18-84%, 数据噪声是关键瓶颈; (2) IEEE S&P 和 USENIX Security 研究证明 LLM 无法可靠识别安全漏洞, 更大模型不优于小模型; (3) 间接提示注入可使 agent 防御降低 30% 以上效用; (4) 越狱在统计意义上不可避免; (5) 模型后门和 AI 生成代码扩展了审计攻击面。本文建立“结论-文献证据-benchmark 样本-评测工件”的追踪矩阵, 结合本仓库 100 样本审计 benchmark 的迭代结果, 提出高可靠审计应从单轮问答转向证据链审计, 从单模型判断转向多阶段系统, 从代码可疑性转向行为可疑性, 并将不确定性和 LLM 不可用显式纳入 fail-closed 机制。

关键词: 大语言模型, 代码审查, 漏洞检测, 程序分析, 静态分析, 检索增强生成, 智能体安全, 越狱攻击, 提示注入, 应用程序审计, 安全软件工程

1 引言

1.1 问题背景

应用程序审计 (application auditing) 指对软件代码、配置、依赖、运行逻辑与外部交互行为进行安全性分析, 以识别命令执行、数据外传、敏感信息泄露、持久化、权限越界、提示注入、工具滥用等风险。传统审计主要依赖人工阅读、规则引擎、污点分析、静态分析和符号/抽象执行等方法。这些方法在既知模式识别上较强, 但在跨文件推理、语义恢复、工程上下文补全和自然语言证据整合方面存在限制。

LLM 的引入改变了审计流程。与只做模式匹配的规则系统不同, LLM 可以解释代码意图、调用链语义、框架惯例和潜在风险目的, 从而在安全审计中扮演语义解释器、候选筛选器、风险裁决器或证据整合器等角色。问题在于, LLM 的“语义能力”并不自动转化为“安全可靠”: 模型可能对可疑模式过度敏感, 导致误报; 也可能在证据不足时给出错误但自信的放行判断, 导致漏报。多项实证研究表明, LLM 在安全漏洞识别中的可靠性仍不足以替代专业审计工具 [14,62,65]: 六个顶级 ML4VD 模型区分漏洞函数与其修复补丁的准确率低于随机猜测 [65], 更大模型反而不如小模型 (LLaMA-3 70B F1=5.10% vs Gemma 7B F1=57.98%) [72], agent 平均 ASR 高达 84.30% 且能力越强的模型越脆弱 [115]。

因此, LLM 在应用程序审计中的最佳角色并非替代传统方法, 而是嵌入多阶段审计体系: 静态分析负责召回, 结构化证据负责上下文压缩, LLM 负责语义裁决, 策略层负责最终阻断或放行, 人工复核处理边界样本。

图 1 展示了代码审计技术从传统方法到 LLM 时代的演进路线。

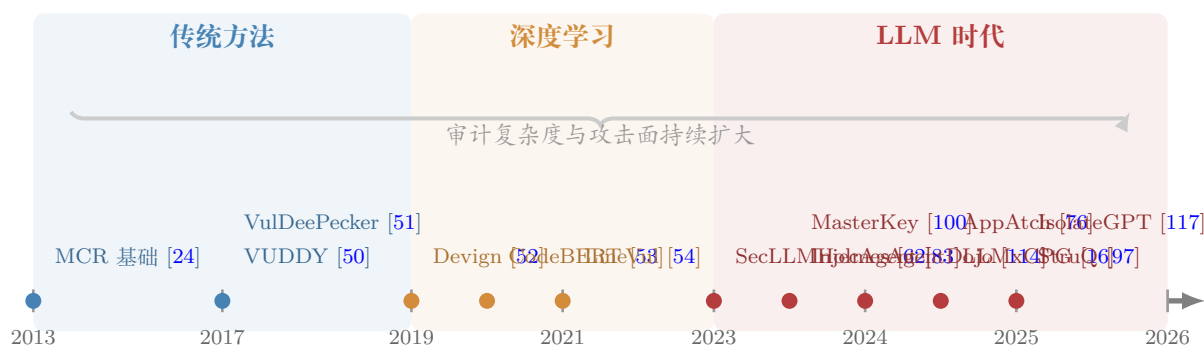


Figure 1: 代码审计技术演进时间线 (2013-2026)

1.2 术语界定

本文区分四个层次：(1) 代码审查，面向函数、补丁、提交或 Pull Request 的局部判断；(2) 漏洞检测，面向可利用缺陷的位置、类型和证据链识别；(3) 程序分析，面向调用关系、数据流、控制流和语义解释的结构化理解；(4) 应用程序审计，覆盖源码、配置、依赖、检索、工具调用、记忆与运行时行为链。本文关注第四层，并将前三层方法视为应用程序审计的组成部分。

1.3 研究问题与贡献

本文围绕四个研究问题展开：

- RQ1: LLM 辅助应用程序审计的主要方法家族有哪些？
- RQ2: 不同方法家族依赖什么证据、解决什么问题、受什么约束？
- RQ3: 在模型选型、prompt 设计、fine-tuning、结构化证据和 benchmark 评估方面，哪些实践更稳定？
- RQ4: 当前文献揭示了哪些共性局限，未来研究方向是什么？

本文贡献如下：

1. 提出面向应用程序审计的七类方法分类，并新增越狱攻防专题；
2. 从证据类型、输出粒度、部署方式和复现约束比较不同方法，覆盖 200 余篇文献；
3. 总结 LLM 审计的常见失效模式与威胁有效性问题；
4. 建立”结论-文献证据-benchmark 样本-评测工件”的追踪矩阵；
5. 给出面向高可靠审计系统的研究议程与工程建议。

2 综述方法

2.1 文献来源与检索范围

本文参考文献覆盖 200 余篇，主要来自 IEEE S&P、USENIX Security、NDSS、NeurIPS、ACL、EMNLP、ICSE、ASE、ISSTA、ACM TOSEM、ACM CSUR、Springer 与 arXiv 预印本，重点覆盖 2023-2026 年发表的工作。检索关键词包括：code review、secure code review、vulnerability detection、vulnerability repair、software security、static analysis、program analysis、retrieval-augmented generation、agent security、prompt injection、jailbreak、malicious skills、benchmark、fair evaluation、secure-aware fine-tuning 与 prompt mixture。

2.2 纳入、排除与证据等级

纳入标准为：与 LLM、代码、安全审计、漏洞检测、程序分析、智能体安全或相关评估方法直接相关；提供方法、系统、综述或实证评估；能够为应用程序审计提供方法论启示。排除标准为：仅讨论通用代码生成而无安全分析内容；与软件安全关联过弱；仅有标题或摘要相关性但缺少方法信息。

为避免把标题相关误认为证据充分，本文将参考文献按证据强度分为三层：A 级为综述或系统综述，用于归纳领域结构与研究空白 [3, 4, 19, 20, 30, 70, 80–82, 124]；B 级为实证或对比研究，用于判断实际可用性与稳定性 [14, 39, 62, 65, 72]；C 级为系统、工具或新方法论文，用于观察工程趋势与技术路线 [16, 54, 58, 114]。

2.3 结构化叙述策略

本文采用结构化叙述综述而非严格 PRISMA 元分析。分析流程为：先按方法家族归类，再按审计对象和证据输入方式细分，最后按可复现性、可解释性、成本与部署约束比较。为降低叙述综述的主观性，本文对每类方法都提出同一组问题：依赖什么证据、解决什么审计子任务、在哪些样本上表现稳定、在哪些边界条件下失败、是否能接入可复现评测流水线。

2.4 文献统计分析

表 1 给出本文引用文献的分布统计。文献主要来自安全领域顶级会议（USENIX Security 52 篇、NDSS 39 篇、NeurIPS 23 篇）和软件工程/自然语言处理会议（ACL 9 篇、ICSE 5 篇、EMNLP 5 篇），以及 arXiv 预印本（26 篇）。时间分布上，2024–2026 年文献占比超过 85%，反映该领域的快速发展。

Table 1: 引用文献分布统计

来源	数量
USENIX Security	52
NDSS	39
NeurIPS	23
arXiv	26
ACL	9
IEEE S&P	7
ICSE	5
EMNLP	5
ACM (TOSEM/CSUR)	4
其他 (Springer, CEUR, etc.)	18
总计	158

2.5 证据追踪与可信度标注

本文把关键结论绑定到四类证据：文献证据、本仓库 benchmarks/audit-100/ 中的样本证据、audit/audit-results/ 中的评测证据，以及 docs/audit/audit-eval-iteration-log.md 中的过程证据。表 2 给出可信度标签。

3 方法家族与代表性工作

图 2 给出本文归纳的十类方法分类体系。方法按审计对象和技术路线分为四大类：代码级分析、结构化增强、系统级安全和评估治理。

Table 2: 结论可信度标签

标签	含义	使用条件
高	文献、benchmark 样本和评测结果相互支持	至少代表性文献 + 样本对照 + 评测记录
中	有文献或样本支持，缺完整复现	可作为设计依据，不宜单独作强结论
低	主要来自方法推断或趋势观察	只能作为研究假设，需实证验证

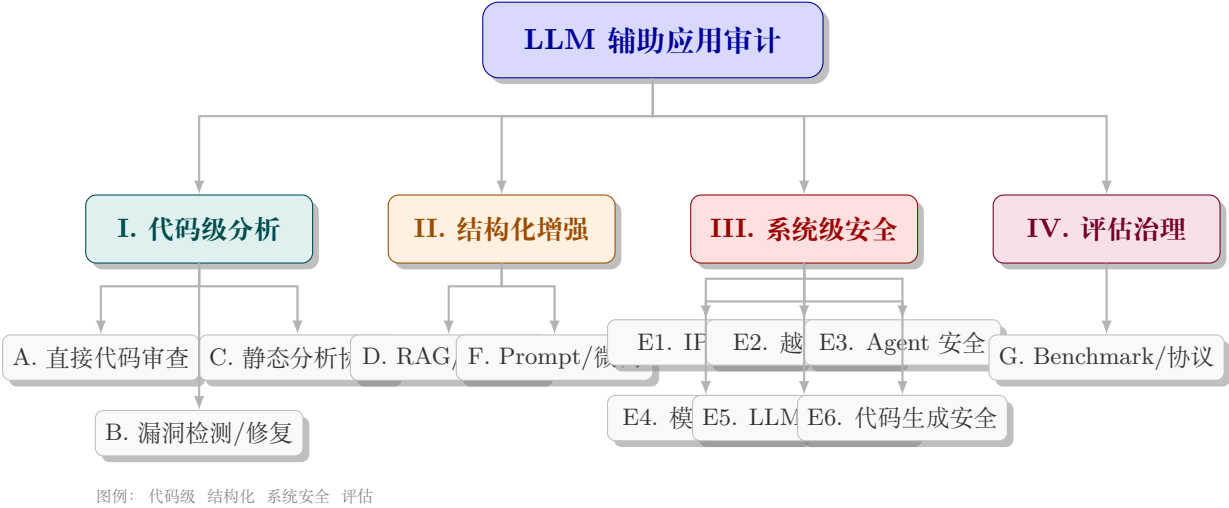


Figure 2: LLM 辅助应用程序审计方法分类体系

3.1 代码审查的演进：从人工到 LLM 辅助

3.1.1 传统代码审查基础

现代代码审查（Modern Code Review, MCR）的研究始于 Bacchelli 和 Bird [24] 在 ICSE 2013 的开创性工作：873 名开发者（44% 响应率）和 165 名经理（28% 响应率）共同表明，缺陷查找是首要动机，但实际产出更偏向知识传递与团队意识；在 570 条评论中，代码改进占 29%，而 91% 的开发者认为文件所有者参与审查有帮助、82% 认为审查整体有价值。McIntosh 等 [25] 进一步证明轻量级代码审查对软件质量的影响因项目而异：Qt 5.0/5.1、VTK、ITK 的缺陷率分别为 19%/14%/9%/11%，但即使有 100% 覆盖，仍有大量后续缺陷暴露，说明覆盖率本身不足以保证质量。Kononenko 等 [26] 对 88 名 Mozilla 开发者的调查显示，审查质量的核心是代码质量与 thorough feedback：49% 的编码引述指向代码质量、38% 指向详尽反馈，且 100% 的受访者认为对代码上下文的熟悉度重要。

在安全视角上，di Biase 等 [27] 对 Chromium 项目中 132,000 条审查评论进行分析后发现，安全相关评论仅约占 1%，在 9,765 条候选安全担忧评论中仅 71 条被人工确认为安全评论；C++ catastrophes 和 buffer overrun 是最常被遗漏的类别，而 command injection 从未在审查中被发现。Sadowski 等 [28] 在 Google 对 9 million reviews 的案例研究展示了审查已服务于缺陷检测之外的多重目的：开发者每周中位数仅审查 4 个变更、初始反馈中位延迟不足 1 小时、总审查中位延迟不足 4 小时，且 70% 的变更在 24 小时内合入。MacLeod 等 [29] 在 Microsoft 的 4,300 人调查与 18 次访谈中揭示，作者端最主要痛点是等待响应，审查端最主要痛点是时间不足。Davila 和 Nunes [30] 对 139 篇论文的系统综述进一步确认：MCR 研究主要依赖开源项目、平均 1–2 名审查者/变更，且静态分析警告只有 6%–22% 会真正进入讨论，安全问题在审查中的可见性仍然偏低。

3.1.2 LLM 时代的代码审查

随着 LLM 的兴起，代码审查自动化进入了新阶段。早期尝试使用神经机器翻译 [31] 和预训练模型 [32, 33] 进行审查自动化，取得了可行性验证但精度有限。Yu 等 [34] 在 OpenStack 和 Qt 社区的研究发现，安全缺陷在代码审查中未被广泛识别，分歧和优先级降低是安全缺陷未被解

决的首要原因。

LLM 时代的代码审查呈现三个特征：第一，多智能体协作架构的兴起。Tang 等 [36] 提出 CodeAgent，利用多个通信智能体协作检测提交信息不一致、漏洞引入和代码风格违规，在 3,545 个真实提交上进行了评估。Rasheed 等 [42] 提出四专家智能体框架，分别关注代码异味、缺陷和改进建议。第二，工业部署的落地经验。Google 的 AutoCommenter [44] (AIware 2024) 已部署到数万开发者，覆盖 C++、Java、Python 和 Go，评论解决率 40%、60% 评论被认为有用，采用 per-URL 阈值和 beam search 覆盖 68% 最佳实践 URL。Google 的 DIDACT V2 系统 [38] (ICSE-SEIP 2024) 通过 ML 辅助解决审查评论，每年自动解决 7.5% 的审查评论 (Java 9.5%、C++ 7.5%)，节省数十万工程师小时。Microsoft 的 WhoDo [37] 在生产环境中部署了审查者推荐系统。第三，LLM 审查的不完美性。Olewicki 等 [40] 在 Mozilla+Ubisoft 的首个活跃用户研究 (59 个审查者、587 个补丁) 中发现接受率仅 8.1%/7.2%，但 23%/28.3% 的评论被认为有价值——接受率低不等于无用。Cihan 等 [46] 发现 GPT-4o 和 Gemini 2.0 Flash 在 492 个 AI 生成代码块上的正确分类率分别仅为 68.5% 和 63.9%。Pornprasit 和 Tantithamthavorn [41] 发现 few-shot learning 可将 GPT-3.5 性能提升至少 46%，但 persona 提示反而降低性能。

Jin 和 Chen [12] 展示了 LLM 代码审查器在需求一致性判断中容易出现“纠错过度”的偏差。Tufano 等 [43] 在 2,291 个预测上使用 120 标签分类学评估，发现 T5 代码审查准确率仅 14.08%/2.11%，且**训练数据集中约 25% 为噪声**——揭示了代码审查自动化的核心矛盾：ChatGPT 难以像人类审查者那样评论代码，专用 DL 模型在审查任务上优于通用 LLM，但数据质量是根本瓶颈。Liu 等 [48] 和 Auto Code Review [11] 进一步证明开源代码审查数据集中包含大量噪声评论，即使经过启发式和 ML 清洗方法仍持续存在，导致模型生成低质量输出。Hong 和 Baik [47] 通过 RAG 结合生成式和检索式方法，缓解生成模型倾向产生高频通用 token 的问题。

在安全审查方面，Peng 等 [45] 的 iCodeReviewer 使用混合提示方法实现 63.98% F1 的安全问题识别率和高达 84% 的审查评论接受率。Charoenwet 等 [39] 的实证研究发现，单个 SAST 工具可在 52% 的漏洞代码变更中标记警告，但 76% 的漏洞函数中的警告不相关，22% 的漏洞贡献提交仍未被检测。Watanabe 等 [35] 的 229 条 ChatGPT 审查评论研究发现，审查者将 ChatGPT 用于外包实现、重构、修复和测试任务，但对审查深度和准确性仍有担忧。Yang 等 [49] 提出了现代代码审查的十年路线图，envisioning 三个范式转变：从被动助手到 IDE 原生主动 AI 协作者、从准确率指标到价值驱动评估、从自主工具到互惠人机反馈循环。

3.2 漏洞检测与修复

3.2.1 深度学习时代的漏洞检测

漏洞检测经历了从规则引擎到深度学习再到 LLM 的演进。Kim 等 [50] 的 VUDDY 提出了可扩展的漏洞代码克隆检测方法，结合函数级抽象和代码相似度在大规模 OSS 生态中发现传播漏洞，实现近零误报。Li 等 [51] 的 VulDeePecker (NDSS 2018) 是首个基于深度学习的漏洞检测系统，在 61,638 个代码片段上达到 TPR=93.0%、FPR=5.7%、F1=90.5% (远超 Flawfinder 27.7% 和 RATS 20.2%)，并发现 4 个此前未知的漏洞。Zhou 等 [52] 的 Devign (NeurIPS 2019) 在代码属性图上构建图神经网络，比基线方法平均准确率提升 +10.51%、F1 提升 +8.68%，对最新 40 个 CVE 的准确率达 74.11%。Feng 等 [53] 的 CodeBERT 提供了通用代码表示的预训练基线。Fu 和 Tantithamthavorn [54] 的 LineVul (MSR 2022) 首次将 Transformer 应用于行级漏洞预测，在 188,000+ 函数上达到函数级 F-measure=0.91 (比基线高 160–379%)，行级 Top-10 准确率 0.65，对 Top-25 最危险 CWE 类型准确率 75–100%。Ahmad 等 [55] 的 PLBART 通过去噪自编码在大规模代码上预训练，为漏洞解释和修复生成提供了基础。

近期工作进一步深化了漏洞检测的技术路线。Mirsky 等 [58] 的 VulChecker (USENIX Security 2023) 在 19 个真实项目中检测到 24 个 CVE (商业工具 Helix QAC 仅检测到 4 个，精度提升 6×)，并发现一个可利用的零日漏洞。Peng 等 [57] 的 PTLVD 将程序切片与 Transformer 结合实现行级检测。Ding 等 [56] 的 VulHawk 实现了跨架构 (x86、ARM、MIPS) 二进制代码漏洞搜索。Woo 等 [59] 的 V1SCAN (USENIX Security 2023) 结合版本匹配和代码相似度，比现有方法多发现 50% 的漏洞 (137 个)，精度 96%、召回 91%，将 FPR 从 71% 降至 4%。Shao 等 [66] 的 FVD-DPM (USENIX Security 2024) 将漏洞检测重构为条件扩散概率生成过程，语句

级定位 IoU 比 Cppcheck 高 +37%，比 DeepLineDP 高 +42%。Shimmi 等 [67] 利用多维邻居嵌入改善跨漏洞类型检测。Liu 等 [61] 使用对比学习解决漏洞 CWE 分类的长尾分布问题。Wang 等 [63] 提出面向任务的知识融合，针对不同代码区域选择性整合异构表示。Chen 和 Xie [60] 利用 LLM 生成受漏洞影响包名称。

3.2.2 LLM 漏洞检测的能力边界

多项实证研究揭示了 LLM 在漏洞检测中的系统性局限。Paquette 等 [62] 在 IEEE S&P 2024 引入 SecLLMHolmes 框架，证明 LLM 无法可靠区分漏洞与非漏洞代码，也不能一致地推理安全属性。Risse 和 Böhme [65] 在 USENIX Security 2024 提出 VulnPatchPairs 基准（26,200 个 C 函数及其补丁对），发现六个顶级 ML4VD 模型在区分漏洞函数与其修复补丁时准确率**低于随机猜测**（< 50%），且数据增强仅使模型过拟合到特定变换——当训练集和测试集使用不同的语义保持变换时，准确率恢复仅 2.5–5.9%。Liu 等 [64] 展示了 ChatGPT 可生成语义保持的对抗代码变体来逃避 DL 漏洞检测器。

Niu 等 [72] 在 NDSS 2025 对 5 个 LLM 家族、21 个模型（7B–70B 参数）的大规模评估揭示：(1) Gemma 7B 在 Java 零样本上达到最高 F1（57.98%），而 LLaMA-3 70B 的 F1 仅 5.10%，**新一代模型反而不如旧版**；(2) GPT-4 的准确率（37.86%）低于开源 Gemma（78.57%）；(3) 上下文窗口扩大一致提升性能（2048→8192 tokens 使 LLaMA-3 8B 准确率从 21.62% 升至 52.70%），但 few-shot learning **反而降低性能**（Gemma 7B F1 从 57.98% 降至 44.60%）；(4) 量化效果因模型而异，无统一最优策略。Khare 等 [71] 发现 LLM 在已知漏洞模式上表现尚可，但在复杂、多步或应用特定漏洞上有效性显著下降。Rossi 等 [74] 发现跨领域迁移时性能显著退化。Zhang 等 [69] 证明精心设计的 prompt 可显著改善 ChatGPT 零样本漏洞检测——链式思维将 C/C++ 准确率从 52.3% 提升至 74.1%，辅助信息（数据流图）将 Java 准确率提升至 74.7%。Song 等 [73] 开发了 Node.js 包的细粒度动态分析。

3.2.3 自动化漏洞修复

漏洞修复是检测的自然延伸。Zhou 等 [70] 的 ACM TOSEM 综述回顾 58 项研究、37 个 LLM，发现最优检测准确率仅 67.6%、修复准确率仅 ~20%，且 63% 修复方法依赖微调、37% 依赖 prompt，6 项关键限制包括函数级输入粒度、缺乏过程间分析和 IDE 集成不足。Zhang 等 [75] 提出第三方库 1-day 漏洞自动补丁方法。USENIX Security 2025 出现三项重要修复工作：Wen 等 [76] 的 AppATCH 使用自适应提示迭代优化 LLM 修复，在零日漏洞上 F1 达 36.46%，无需利用证据即可超越传统 APR 工具；Kim 等 [77] 的 SAN2PATCH 仅用 sanitizer 日志和源码结合思维树推理，在真实漏洞上达到 79.5% 正确补丁率；Yu 等 [78] 的 PatchAgent 实现端到端自主修复，修复成功率 92.13%，从定位到验证无需人工干预。Zhou 等 [2] 强调“发现漏洞”和“生成补丁”不是同一件事。SoK [79] 系统化整理了自动修复全景，将方法分为规则、学习和 LLM 三类，识别了过度依赖合成 benchmark 的评估陷阱。

3.2.4 综合综述

近期多篇综述系统化了 LLM 安全检测领域。Kaniewski 等 [82] 的 ACM TOSEM 综述分析了 263 篇研究（2020–2025），是该领域最大规模的 SLR，发现研究高度碎片化——各研究使用自定义数据集、指标和数据划分，难以跨研究比较。Sheng 等 [80] 的 ACM CSUR 综述涵盖 80+ 篇论文和 33 个不同 LLM，发现 GPT-4 是最常用模型（29 次使用），系统化了提示策略、微调方法和评估方法论，识别了幻觉、数据污染和可复现性等关键挑战。Chen 等 [81] 的 ACM TOSEM 综述系统回顾了 68 篇关于 CodeLM 安全性的论文，涵盖后门攻击和对抗攻击两大类。Zhang 等 [124] 的综述涵盖 300+ 项工作、25 个 LLM 和 10+ 下游场景。各综述一致指出：高质量数据集稀缺是首要瓶颈，混合方法（LLM + 传统分析）最具前景，仓库级分析仍未解决。

3.3 LLM 与静态分析协同

静态分析协同是当前最具工程可行性的路线之一。其基本思想是：静态规则、污点分析或图分析负责提供可重复、可解释的候选证据，LLM 负责判断这些证据在当前上下文中是否真正构成安

全威胁。相关研究包括 LLM 辅助程序分析综述、静态分析与 LLM 协同、CPG 引导的漏洞检测，以及 LLM 与静态工具的系统 benchmark [5, 15, 16, 18]。

该路线适合应用程序审计，因为审计同时需要高召回和低误报。静态工具负责发现可疑点，LLM 负责解释为什么可疑或为什么不危险。优势是可追溯、可审计、可接入 CI；局限是前处理链条更长，且静态证据缺失会使 LLM 只能在残缺上下文中判断。Charoenwet 等 [39] 的实证数据支持了这一判断：SAST 工具对漏洞代码变更的覆盖率仅 52%，且 76% 的警告不相关，说明 LLM 在过滤误报方面有显著价值。

例如，LLMxCPG [16] 以 CPG 为中心组织程序上下文，利用图结构和切片把漏洞候选压缩成更高信噪比的证据；Synergizing [15] 则直接把“静态分析 + LLM”组合成混合工作流；LLM vs Static [18] 在 10 个 C# 项目（63 个嵌入漏洞）上的系统比较证实二者互补而非替代：GPT-4.1 F1=0.797、Mistral Large F1=0.753、DeepSeek V3 F1=0.750，远超 SonarQube（F1=0.260）、CodeQL（F1=0.386）和 SnykCode（F1=0.546）——LLM 优势来自更宽的代码上下文召回，但所有 LLM 因 tokenization 在行/列定位上存在偏差；推荐混合流水线：LLM 负责早期广域分诊，确定性扫描器负责高保证验证。Kaniewski 等 [82] 在工业流水线评估中发现，LLM 在定位和解释方面表现优秀，但严重性校准仍需人工确认。

3.4 检索增强与结构化表示增强

结构化/RAG 路线的核心思想是：不要把整份源码直接交给模型，而是先将程序知识压缩为适合审计的证据，再让 LLM 在证据上推理。常见输入包括 CPG、CFG、DFG、调用链、数据流路径、敏感源/汇点、API 文档、历史提交、相似漏洞案例和项目规范 [5, 15, 16]。

结构化表示的收益是缓解上下文窗口限制、提升证据密度、让模型围绕“为什么危险”而不是“代码长什么样”判断；局限是图构建和切片本身可能出错，关键边或节点缺失会直接影响结论。RAG 的收益是补充 API 语义和项目约定；风险是检索到的外部文本可能含有噪声、误导说明或提示注入。Hong 和 Baik [47] 通过 RAG 缓解代码审查中生成模型的通用 token 倾向。Tan 等 [99] 的 PRA-RAG 提供了首个可证明鲁棒的 RAG 聚合机制，通过约束被污染检索文档对最终输出的影响，将攻击成功率从基线的 >40% 降至 ~1%，同时在干净数据上保持 ~71% 准确率——是目前唯一同时提供理论保证和实验验证的 RAG 防御方案。

Table 3: 结构化/RAG 审计可靠性评估维度

层次	典型失败模式	对结论的影响
表示构建	动态导入、反射导致调用边缺失	恶意链被切断，形成漏报
证据切片	只保留危险 API，不保留校验条件	误报或无法证明可利用性
检索质量	相似但无关案例进入上下文	LLM 被错误先验牵引
检索投毒	对抗性文档注入恶意指令	IPI，审计结论被篡改
证据冲突	文档声称安全，代码路径显示外传	应升级为 UNCERTAIN 或人工复核
可复现性	检索源、摘要、排序变化	指标不可比较，结论难复核

3.5 面向智能体与应用执行链的审计

随着 LLM 从代码生成器扩展为 agent，审计对象不再只是静态源码，而是会检索、调用工具、规划行动并在多轮交互中改变状态的执行系统。2024–2026 年间，agent 安全已成为独立研究领域，涵盖 prompt injection 攻防、jailbreak 攻防、agent 行为链安全和模型完整性等多个方向。图 3 给出 agent 安全的攻防全景。

3.5.1 间接提示注入攻防

间接提示注入（IPI）是 agent 安全的核心威胁。Zhan 等 [83] 的 InjecAgent（ACL 2024）引入 1,054 个测试用例的 benchmark（17 用户工具 × 62 攻击工具），发现 ReAct-prompted GPT-4 ASR 为 24%（增强攻击下升至 47%），Llama2-70B 超过 80%，但微调模型显著更鲁棒——微

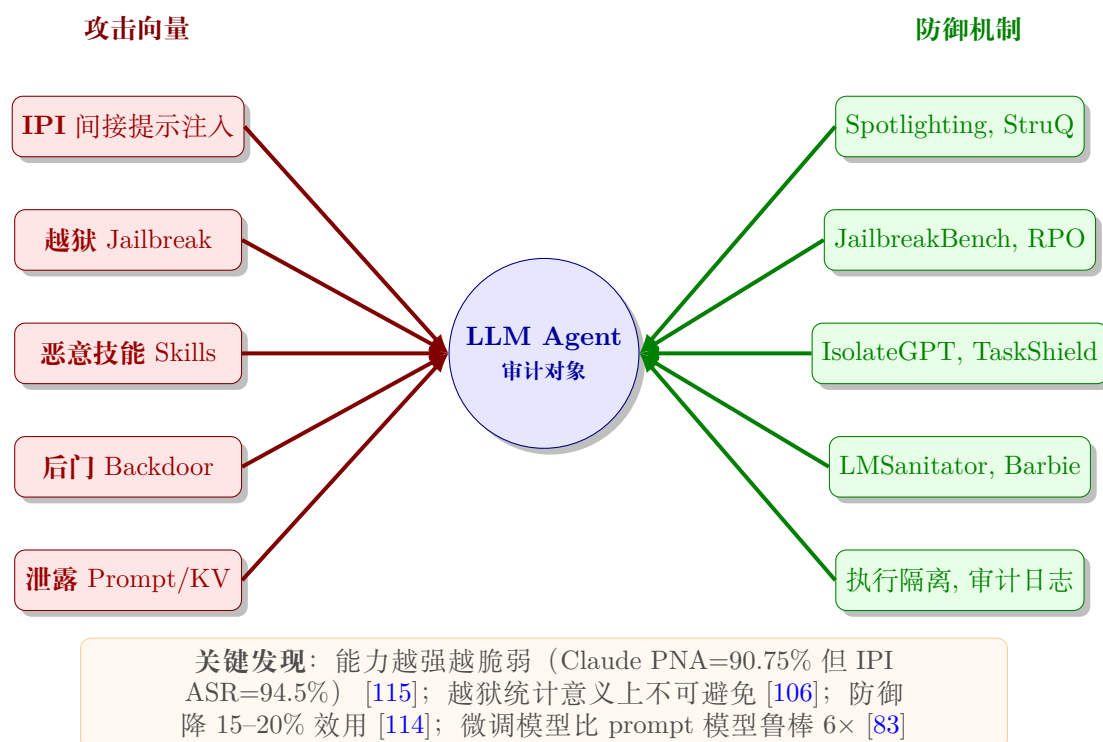


Figure 3: Agent 安全攻防全景图

调 GPT-4 ASR 仅 3.8%，远低于 prompt 版本的 24%。Liu 等 [84] (USENIX Security 2024) 提出统一形式化框架，在 10 个 LLM、7 个任务上评估 5 种攻击和 10 种防御——其 Combined Attack 在 10 个 LLM 上平均 ASV=0.62、MR=0.78，当目标任务和注入任务类型相同时检测极其困难，没有单一防御普遍有效。Li 等 [86] 揭示了一个根本性张力：指令跟随能力越强的模型反而越容易被提示注入，因为它们更倾向于遵从注入指令而非原始指令。Kong [85] 开发了系统化的 IPI benchmark 框架。Chen 等 [87] 发现检测分类器在独立设置中准确率高，但在端到端 agent 场景中因分布偏移而显著退化。

防御方面，Hines 等 [90] 的 Spotlighting 通过不可见标记区分可信指令与不可信数据——datamarking 将 GPT-3.5 摘要任务 ASR 从 50% 降至 3.10%，编码方案降至 0%，且不损害 NLP 任务性能。Ayub 和 Majumdar [91] 在 467,057 条 prompt 上证明基于嵌入的轻量分类器可以高准确率检测提示注入——Random Forest + OpenAI 嵌入达 AUC=0.764、F1=0.868，在 AUC 和精度上超越 4 个 DeBERTa SOTA 分类器（DeBERTa 召回 >0.91 但精度仅 0.75–0.77）。Chen 等 [97] 的 StruQ 在 API 层面强制 prompt 与数据分离，对 Naive/Ignore/Completion-Real 等攻击实现 ASR 从 96% 降至 0%，TAP 从 97% 降至 9%，同时保持 AlpacaEval 效用不变（67.2%→67.6%）。Jia 等 [95] 的 Task Shield 通过任务对齐约束阻止 IPI 导致的偏离。Wen 等 [96] 的 InstructDetector 利用 LLM 行为状态（隐藏层状态 + 梯度）检测 IPI 攻击，仅需 200 个训练样本即达 99.60% 域内精度和 96.90% 域外精度，在 BIPIA benchmark 上将 GPT-3.5 ASR 从 33.57% 降至 0.12%、GPT-4o 从 39.68% 降至 0.13%。Chen 等 [94] 用攻击技术反制攻击，通过对抗性增强训练提高鲁棒性。Tan 等 [99] 的 PRA-RAG 提供了首个可证明鲁棒的 RAG 聚合机制。Ramakrishnan 和 Balaji [98] 提出多层防御框架，将 IPI 成功率降低超 70%。

Evertz 等 [92] 揭示 LLM 集成系统中的机密性威胁：Llama 3.1 在无防护时 99% 泄露密钥，显式“不要泄露”指令仍泄露 21–34%，越狱恢复至 94–97%；Context Ignoring 是最有效攻击（8b 上 91% ASR）；工具集成可将提取成功率提升 73%。Kim 等 [88] 证明上下文长度（而非 shot 数）是 many-shot 攻击有效性的主要决定因素。Evtimov 等 [89] 的 WASP benchmark（84 个对抗任务）展示了网页导航中 prompt injection 可劫持商业 agent——中间劫持成功率高达 85.7%（o1 模型），但端到端目标完成率仅 0–16.7%，揭示“安全源于无能”现象：攻击可劫持 agent 但难以完成复杂攻击目标。URL 注入比纯文本注入有效 2.6×（GPT-4o 上 61.9% vs 23.8%），防御性系统提示可将 ASR 从 42.9% 降至 22.6%。

3.5.2 越狱攻击与防御

越狱攻击旨在绕过 LLM 的安全对齐。Deng 等 [100] 的 MasterKey (NDSS 2024) 利用基于时间的防御探测技术逆向工程聊天机器人防御机制, 自动生成越狱 prompt, 平均成功率 21.58% (基线的 3 $\times$), 并发现越狱 prompt 具有 GPT 专属性——对 Bard 和 Bing Chat 的迁移率接近零。Yu 等 [101] 对野外越狱 prompt 进行大规模实证研究, 分类出角色扮演、对抗后缀和逻辑推理链三种核心攻击机制。Yu 等 [102] 将软件测试的 fuzzing 方法引入越狱发现, 识别出数千个未知攻击向量。

NeurIPS 2024 集中发表了多项越狱研究。Xu 等 [103] 将越狱分解为 token 级和 prompt 级组件, 发现语义混淆和角色扮演框架主导字符级扰动。Hu 等 [104] 的 ADC 实现从密集到稀疏的 token 级优化, 查询效率远超先前方法。Zheng 等 [105] 证明精心构造的 few-shot 演示可跨模型迁移并绕过已知防御。Su 等 [106] 从统计学习角度证明越狱在理论上不可避免: 任何有限训练集上的对齐都留有可利用的残余漏洞。

USENIX Security 2025 继续推进: Gong 等 [107] 的 PAPILLON 结合 fuzz testing 和 LLM 引导变异生成语义连贯的越狱 prompt, 对 GPT-3.5 实现 >90% ASR、GPT-4 >80%、Gemini-Pro >74%, 生成 prompt 的困惑度仅 34.61 (远低于检测阈值 58.83), 绕过 perplexity-based 检测器的能力显著优于基线。Krauss 等 [108] 的 TwinBreak 同时嵌入良性和恶意请求, 利用 LLM 处理第二条指令的倾向, 在 16 个模型 (1B–72B 参数) 上实现 89–98% ASR, 运行时间 <5 分钟, 工具损失 $\leq 4.5\%$ 。Zhang 等 [109] 识别了代码生成、翻译等任务级漏洞。

防御方面, Chao 等 [110] 的 JailbreakBench 建立了标准化可复现评测协议, 揭示许多已发表防御在标准化测试下提供的鲁棒性微不足道——其 Prompt+RS 基线对 4 个 LLM 分别实现 89/90/93/78% ASR, 其中 Erase-and-Check 是最强防御但仍难以完全阻断。Han 等 [111] 的 WildGuard 发布开放审核工具包, 联合执行恶意意图检测、响应安全分类和拒绝检测。Jiang 等 [112] 从真实交互中挖掘 5,700 个新越狱策略, 对抗性训练产出更鲁棒模型而不降低有用性。Zhou 等 [93] 的 RPO 自动生成鲁棒指令前缀, 提供 Lipschitz 有界威胁模型下的可证明鲁棒性保证。Gong 等 [113] 揭示对齐技术 (RLHF、DPO) 产生“对齐税”——HarmBench ASR 从 42% 降至 36.95%、AdvBench 从 28.40% 降至 20.89%, 但在某些维度反而降低安全性。

3.5.3 Agent 系统安全

Agent 安全审计关注“输入–决策–工具–输出”的完整链路。Debenedetti 等 [114] 的 AgentDojo (NeurIPS 2024) 引入动态评估环境 (97 个任务、629 个安全测试、4 个环境、74 个工具), agent 在不可信数据上执行真实工具调用——最佳 agent 的良性效用天花板仅 66%, Slack 套件 ASR 高达 92%, 工具过滤防御可将 ASR 降至 7.5% 但 17% 测试用例仍然失败, 所有防御均导致 15–20% 效用损失。Zhang 等 [115] 的 Agent Security Bench (ICLR 2025) 覆盖 10 个场景、400+ 工具、13 个 LLM 骨干、13 种攻击和 11 种防御, 发现平均 ASR 高达 84.30% (PoT 后门攻击对 GPT-4o 达 100%), 防御平均有效性仅 46.91%, 且**能力越强的模型越脆弱**——Claude 3.5 Sonnet 效用最高 (PNA=90.75%) 但 IPI ASR 也达 94.50%。Zhang 等 [116] 的 Agent Audit 检查模型权重之外的工具代码、部署配置和权限设置。

Wu 等 [117] 的 IsolateGPT 提出 hub-and-spoke 执行隔离架构, 每个应用运行在独立 spoke 中配备专用 LLM 实例, 在 1,598 个攻击场景上将 VanillaGPT 的 81.3% 跨应用攻击成功率大幅降低, 75.73% 查询性能开销 <30%, 同时保持 100% 功能完整性。Ayzenshteyn 等 [118] 的 CHaT 框架使用 Stackelberg 博弈建模的欺骗 (蜜罐) 和反击策略, 在 11 台 CTF 机器上实现 100% 防御成功率 (0/3 被攻破), 累积 5 个随机陷阱后 DSR >95%, 利用 LLM 的训练偏差、DFS 搜索倾向和幻觉弱点。Liu 等 [119] 的 AgentFuzz 将程序安全的污点分析和有向灰盒模糊测试适配到 LLM agent, 在 20 个开源 agent 应用上发现 34 个 0-day 漏洞 (23 个 CVE), 精度 100%、召回 97.14%, 比 LLMsSmith 精度提升 33 $\times$ 。Liu 等 [9, 120] (USENIX Security 2026) 对 98,380 个社区 agent 技能的分析发现 157 个确认恶意技能 (0.16%), 包含 632 个漏洞覆盖 6 个 MITRE ATT&CK 杀伤链阶段, 每个恶意技能平均 4.03 个漏洞, >50% 源自单一威胁行为者。Agent Audit [6] 提出 agent 感知静态分析框架, 在 42 个标注漏洞上达到 F1=0.909 (Semgrep 0.385、Bandit 0.458), 召回率 95.24%, 比 Semgrep 高 4 $\times$, MCP 漏洞覆盖率 100%。Wu 等 [121] (NDSS 2025) 的 PromptPeek 利用 KV-cache 共享侧信道攻击, 在多租户 LLM 服

务中以 96–100% 成功率提取其他用户的 prompt, GQA 架构使攻击更容易 (100% 全场景)。

Xu 等 [8,122] (AI Review 2026) 通过“双重性”框架综述 agent 安全 (89 页), 揭示仅 0.1% 数据投毒即可实现 >80% 后门攻击成功率且对良性性能影响 <1%, 即使最安全的 agent 仍有 23.9% 测试用例存在失败风险。Khonde 等 [7,123] 提供 RAG agent 管道的端到端安全综述, 涵盖检索投毒、IPI 和工具攻击三类威胁。Chen 等 [87] (ACL 2025) 构建 Inj-SQuAD/Inj-TriviaQA 基准, 发现通用 LLM 检测 IPI 准确率仅 42–79%, 但专门训练的 DeBERTa 达 99.12%, Segment-DeBERTa 组合过滤可 100% 移除注入指令, 将 StruQ 的 ASR 进一步降至 0.11%。Ramakrishnan 和 Balaji [98] 在 847 个对抗测试上验证三层防御框架, 将整体 ASR 从 73.2% 降至 8.7%, 延迟开销仅 2.1%。Prompt injection agents [17] 强调间接提示注入可绕过主模型的表面安全。

3.5.4 模型完整性与后门威胁

当审计系统依赖微调模型或第三方预训练模型时, 模型本身的完整性成为审计可信性的前提。后门攻击可在模型中植入触发器, 使特定输入产生攻击者期望的输出, 从而破坏审计判断。

2023–2025 年间, 后门攻防研究呈现多样化演进。Beatrix [126] 使用 Gram 矩阵进行鲁棒后门检测。ASSET [127] 在多重 DL 范式下实现鲁棒后门数据检测。PELICAN [128] 揭示自然训练模型中可利用的后门在二进制代码分析中的存在。VILLAIN [129] 针对垂直联邦学习发起后门攻击。Trojan Source [130] 展示了不可见字符可产生源码级别的隐形漏洞。

与 LLM 审计直接相关的是: Wei 等 [131] 的 LMSanitizer (NDSS 2024) 在 960 个模型上达到 92.8% 后门检测准确率, 将 ASR 降至 <1%, 且仅降低 0.075% 准确率 (远优于 ONION 的 2.2%), 揭示 prompt-tuning 容易受到预训练模型中任务无关后门的影响。Zhang 等 [132] (USENIX Security 2024) 展示针对定制化 LLM (如 OpenAI GPTs) 的指令后门攻击, 词级触发器在 6 个 LLM 上均达到 ASR=1.000, 但防御端 5.8% 误报率意味着 300 万个 GPTs 中约 17.4 万个会被误标。BackdoorAlign [134] (NeurIPS 2024) 仅用 11 条安全样本即将 Llama-2 ASR 从 94.91% 降至 3.64%、GPT-3.5 从 75.64% 降至 14.91%, 比基线防御节省 27× 数据量。Philosopher’s Stone [135] 展示 LLM 插件中的木马化风险。Yan 等 [133] 进一步证明 LLM 可辅助对代码补全模型发动后门攻击, 扩大后门注入的攻击面。

防御方面, Beatrix [126] (NDSS 2023) 使用 Gram 矩阵在动态后门检测上达 F1=91.1%, 远超 SOTA 的 36.9%。Barbie [136] (NDSS 2025) 基于潜在可分性在 >10,000 个模型上验证, 对 14 类后门攻击的 TPR 接近 100%、FPR <3.65%, 比 FreeEagle 在样本特定攻击上提升 +43% TPR。CLiBe [137] 检测 Transformer NLP 模型中的动态后门。DShield [138] (NDSS 2025) 通过差异学习同时防御脏标签和干净标签 GNN 后门攻击, 将 Cora 上 ASR 从 97.81% 降至 0.74%。这些工作共同表明: 若审计管线中使用的 LLM 或嵌入模型本身被后门化, 则所有下游审计结论均不可信。因此, 高可靠审计系统必须将模型完整性验证纳入信任链。

3.5.5 LLM 增强安全测试

LLM 不仅被审计, 也在主动参与安全测试。Deng 等 [139] 的 PentestGPT (USENIX Security 2024) 将 LLM 用于自动化渗透测试, 在 13 个目标上将任务完成率提升 228.6% (GPT-4 完成 5/13 目标), picoMini CTF 排名 24/248, 但上下文丢失是主要失败原因 (195 次试验中 74 次)。Asmrita 等 [140] (USENIX Security 2024) 在 293 个 BusyBox ELF 二进制上利用 LLM 生成种子和崩溃重用技术, 无需传统模糊测试即在最新 BusyBox 中发现 97 个崩溃 (19 个唯一)。DeepGO [141] (NDSS 2024) 使用预测引导的有向灰盒模糊测试, 到达目标站点速度比 AFLGo 快 3.23×, 暴露 19/20 个已知漏洞。EnclaveFuzz [142] (NDSS 2024) 在 14/20 个 SGX enclave 中发现 162 个 bug, 代码覆盖率 16.53% (SGXFuzz 仅 4.57%)。LLM 引导的协议模糊测试 [143] 从 RFC 规范自动生成结构化种子和状态感知变异策略。

在传统安全测试方面, Fuzztruction [159] (USENIX Security 2023) 通过对生成器而非输入进行故障注入, 在复杂格式消费者上比 AFL++ 等工具平均多 21% 覆盖率 (部分目标达 70%), 发现 151 个唯一崩溃和 4 个 CVE。NAUTILUS [161] (USENIX Security 2023) 针对 RESTful API 的多步漏洞检测, 比现有扫描器多发现 141% 漏洞, 在真实服务中发现 23 个零日漏洞 (12 个 CVE), 包括 Atlassian Confluence 远程代码执行和 Microsoft Azure 高危漏洞, 揭示 82% 的 API 漏洞需跨多个操作才能触发。ICSPatch [160] (USENIX Security 2023) 在 24 个 ICS 控制

二进制上实现 100% 漏洞定位成功率和 $\sim 124\text{--}222\text{ ms}$ 补丁生成时间, 运行时开销仅 $0.05\text{--}17.4\text{ }\mu\text{s}$, 为工业控制系统提供零停机热补丁。在侧信道方面, CipherH [162] (USENIX Security 2023) 在 OpenSSL、MbedTLS、WolfSSL 中检测到 236 个密文侧信道漏洞点 (精度 $\sim 96.2\%$), 发现 WolfSSL 中此前未知的漏洞并推动官方发布补丁。CacheQL [163] (USENIX Security 2023) 使用 Shapley 值量化缓存侧信道泄露量 (如 OpenSSL AES-128 泄露 128 比特), 对 13 个常量时间函数正确报告零泄露, 验证了精度。

在代码安全增强方面, DeGPT [144] 使用 LLM 优化反编译器输出, 改善逆向工程中的代码可读性。SigmaDiff [145] 通过语义感知深度图匹配实现伪代码差异分析。GNNic [146] 利用抽象相似度查找“失散”的函数兄弟关系。

在 API 安全方面, Midas/ChatDetector [147] (NDSS 2025) 以 98.21% 精度识别 165 个 RM-API 对, 发现 115 个安全 bug, 链式思维 + 交叉验证将精度从 77.97% 提升至 98.21%。API 参数安全规则生成 [148] 使用 LLM 生成 API 误用检测的安全规则。NodeMedic-FINE [73] 对 Node.js 包进行细粒度漏洞检测和利用合成。YuraScanner [149] (NDSS 2025) 利用 LLM 驱动任务导向的 Web 应用扫描, 在 20 个 web 应用上发现比其他工具多 55.19% 的表单和 35.16% 的 URL, 发现 12 个零日 XSS 漏洞。

3.5.6 LLM 代码生成安全

LLM 生成的代码本身可能引入安全缺陷, 这使得审计不仅面向手写代码, 也面向 AI 生成代码。Ullah 等 [150] (IEEE S&P 2024) 在 228 个代码场景、8 个 LLM 和 17 种提示技术上评估发现: 改变函数名导致 PaLM2 在 26% 情况下给出错误答案, 添加库函数导致 GPT-4 在 17% 情况下失败, 且所有模型即使在温度 $=0.0$ 时仍存在非确定性输出。Niu 等 [151] (USENIX Security 2023) 发现约 8% 的 prompt 可从 Codex 模型中泄露隐私信息 (邮箱、姓名、SSN、API 密钥)。

INDICT [152] (NeurIPS 2024) 在代码生成中引入双重批评者内部对话机制, 在 7B–70B 参数的 8 种语言上实现 +10% 代码质量绝对提升, HarmBench 安全分类从 23.6% 提升至 51.5%。LeDex [153] 训练 LLM 更好地自调试和解释代码。WorldCoder [154] 通过构建世界模型和编写代码评估 LLM agent 能力。Verified Code Transpilation [155] 使用 LLM 进行经过验证的代码转译。

REPOCOD [156] (ACL 2025) 在 980 个仓库级 Python 任务上发现**没有 LLM 超过 30% pass@1** (GPT-4o 最高仅 27.4%), 50.8% 任务需要仓库级上下文。Can You Trust Code Copilot [157] (ACL 2025) 在 20 个 LLM 上发现: 训练数据中 4.4% 包含漏洞, 与 LLM 生成代码的漏洞分布高度相关; Claude-3 漏洞修复率 66.25%、GPT-4o 63.94%, 但大多数 LLM 检测 $F1 > 80\%$ 却仍生成含漏洞代码。SecRepoBench [158] (LLM4Code 2026) 在 318 个 C/C++ 仓库级补全任务上发现最强独立模型 GPT-5 仅 39.3% secure-pass@1, 代码 agent 显著提升正确性但安全性增益有限 (仅 +4.7%)。

这些工作共同说明: LLM 代码生成是审计的新攻击面, 审计系统需要能够检测 AI 生成代码中的安全缺陷, 且不能假设 AI 生成代码比手写代码更安全。

3.6 Prompt 设计、安全感知微调与交互策略

Prompt 与 fine-tuning 是审计性能的重要决定因素。同一模型在不同提示模板下输出差异显著, 安全任务中少量措辞变化即可影响误报/漏报 [1, 10, 21, 22, 41, 69]。Niu 等 [72] 在 NDSS 2025 的系统评估中明确发现: **prompt 设计比模型规模更重要**——7B 参数模型在精心 prompt 下可超越 405B 模型的零样本性能。

安全审计 prompt 不是让模型自由发挥, 而是强约束式判定协议, 应包含以下要素:

- **输出类别约束**: 明确枚举 benign/malicious/uncertain, 禁止模糊回答
- **证据优先级**: 要求模型先列出证据再给结论, 而非先结论后解释
- **Benign trap 豁免**: 显式说明教学示例、测试用例、包装代码不应被标记为恶意
- **JSON 格式约束**: 强制结构化输出, 便于自动化解析和聚合
- **不确定性处理**: 明确定义 UNCERTAIN 的触发条件 (证据不足、多证据冲突)

在微调策略上，Mixture-of-prompts [22] (华为，ASE 2025) 将不同样本交给不同提示策略并结合静态分析路由，实现 63.98% F1 和 84% 生产接受率 (C/C++)。Secure-aware fine-tuning [21] (ICSE 2026) 将安全判断偏好嵌入模型参数，结合 RAG 达到 F1=71.98% 和 SecureBLEU=29.31。Chen 等 [81] 的大规模实证研究发现任务特定微调一致优于通用预训练模型。Pornprasit 和 Tantithamthavorn [41] 发现 few-shot learning 可将 GPT-3.5 性能提升至少 46%，但 persona 提示反而降低性能。Effectiveness [13] 从实证评测角度说明模型即使能识别可疑模式，也未必能稳定完成漏洞成因推理——识别和推理是两个不同层次的能力。

Zhang 等 [69] 证明精心设计的 prompt 可显著改善 ChatGPT 零样本漏洞检测，但改善幅度高度依赖漏洞类型：CWE-79 (XSS) 等模式化漏洞改善明显，而 CWE-502 (反序列化) 等需要深层语义理解的漏洞改善有限。这进一步支持了本文的核心主张：prompt 优化是必要但不充分的条件，高可靠审计需要系统级架构而非单轮问答。

3.7 Benchmark 设计与评估协议

LLM 辅助审计的研究质量高度依赖 benchmark。2024–2025 年见证了 benchmark 设计的爆炸式增长。图 4 给出主要 benchmark 对不同威胁维度的覆盖情况。

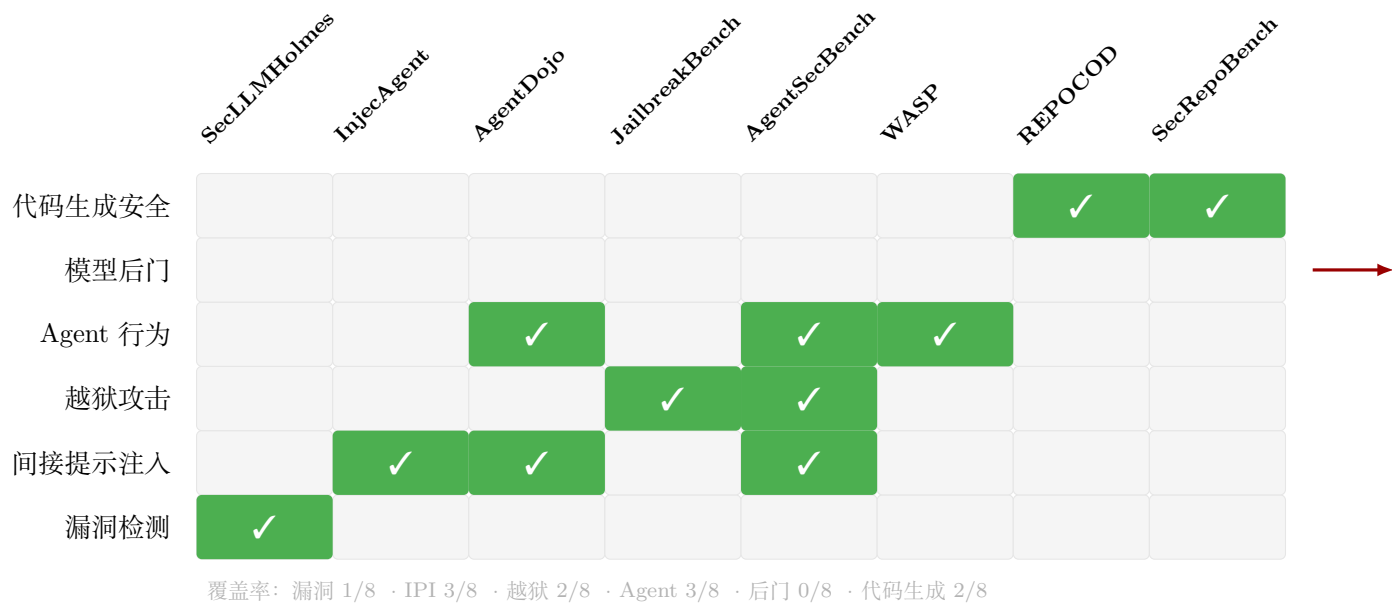


Figure 4: 主要 Benchmark 对威胁维度的覆盖矩阵

3.7.1 漏洞检测 Benchmark

LLM vs Static [18] 将 LLM 与静态工具放在同一基准比较。Ullah 等 [14] 和 Paquette 等 [62] 分别从推理和识别两个角度建立了 LLM 漏洞检测评测框架。Zhang 等 [68] 提出 VulnLLMEval 统一评估框架。Fair Security Eval [23] 强调公平评估原则。Niu 等 [72] 首次对 7B–405B 规模模型进行系统对比。

3.7.2 Agent 安全 Benchmark

InjecAgent [83] 和 InjectBench [85] 聚焦间接提示注入。Liu 等 [84] 提供统一形式化和综合 benchmark。AgentDojo [114] 和 Agent Security Bench [115] 覆盖完整 agent 安全场景。WASP [89] 关注 Web agent 安全。Zhang 等 [109] 提供任务级越狱评估。

3.7.3 越狱 Benchmark

JailbreakBench [110] (NeurIPS 2024) 建立标准化评测协议 (100 个行为、4 种攻击基线、5 种防御基线), 其 Prompt+RS 基线对 Vicuna/Llama-2/GPT-3.5/GPT-4 分别实现 89/90/93/78% ASR, Erase-and-Check 是最强防御但 Prompt+RS 仍能达 8–25%; GCG 需 25.6 万次查询和 1700 万 token, 而 Prompt+RS 仅需 2–1K 次查询。Bag of Tricks [103] 分解攻击组件, 发现语义混淆和角色扮演主导字符级扰动。MasterKey [100] 实现自动化越狱评估并发现越狱 prompt 的 GPT 专属性。PAPILLON [107] 和 TwinBreak [108] 将攻击成功率推到 89–98% 范围。LLM-Fuzzer [102] 将 fuzzing 方法论引入评测。

3.7.4 代码审查 Benchmark

CRUXEval [125] (ICML 2024) 在 800 个 Python 函数上评测代码执行推理能力: GPT-4 在输入预测上 pass@1=67.1%、输出预测 63.4%, 链式思维提升至 75%/81%; 但最佳开源模型仅 47.9%/44%, 且链式思维对开源模型**无改善**——揭示闭源与开源模型在代码推理上的巨大鸿沟。Peng 等 [45] 报告 63.98% F1 的安全审查基准。Cihan 等 [46] 在 492 个 AI 代码块上评测 LLM 审查能力。

4 综合比较

Table 4: 方法家族综合比较矩阵

家族	核心目标	证据输入	主要优势	主要短板	本仓库样本	可 信 度
直接代码审查	局部安全判断	函数、补丁、PR	接入简单	上下文弱, 18–68%	B1-08 vs M1-01	高
漏洞检测/修复	定位并修复	代码片段、上下文	任务边界清楚	ML 学表面模式	eval(label) vs eval()	高
静态分析协同	召回 + 裁决	规则命中、调用链	可追溯、入 CI	SAST 76% 不相关	M3-02 vs B3-01	高
结构化/RAG	高信噪比证据	CPG/CFG、切片	降噪、补上下文	前处理和检索污染	M5-01	中–高
Agent 审计	运行时行为	prompt、工具、记忆	覆盖真实攻击面	能力越强越脆弱, 防御降 15–20%	行为链类比	中
越狱攻防	鲁棒性评估	对抗 prompt	系统化评测	89–98% ASR, 理论不可避免	N/A	中
模型完整性	信任链验证	权重、激活	确保审计器可信	后门化 → 全失效	N/A	中
LLM 安全测试	主动漏洞发现	LLM fuzzing	扩大覆盖面	依赖 LLM 能力	N/A	中
代码生成安全	AI 代码审计	生成代码	覆盖新攻击面	AI 代码引入缺陷	N/A	中
Prompt/微调	稳定判定	约束 prompt	改善一致性	版本标签敏感	B3-01/B3-04	中
Benchmark	评估有效性	manifest、样本	可复跑比较	来源泄漏风险	audit-100	高

4.1 审计器模型与使用方式

本文严格区分被审计代码包与审计器模型。LogisticRegression、TEE-test、Retina-DKD 是 subject-side 样本来源; Claude、GPT、Gemini、Qwen、DeepSeek、StarCoder 等才是 auditor-side

模型候选。表 5 给出审计器类型比较。

Table 5: 审计器模型与使用方式对比

类型	代表模型	推荐角色	优势	风险	状态
前沿闭源	Claude、GPT-4.1、Gemini	语义裁决、边界仲裁	推理强，跨文件	成本、版本漂移	候选
本地代码	Qwen2.5-Coder、DeepSeek	可复现批量审计	成本低，可离线	格式不稳	已接入
静态规则	Python/Go 规则	高召回、fail-closed	可解释、可版本化	易被变形绕过	满分
混合系统	静态 + 结构化 + LLM+ 策略	生产级审计闭环	兼顾召回与追溯	工程最复杂	目标

4.2 当前评测结果与含义

表 6 汇总本仓库关键评测阶段。

Table 6: 当前仓库评测阶段对比

阶段	样本语义	代表结果	指标含义	后续动作
初始静态	来源未隔离	TP=43, FP=22, TN=28, FN=7	有召回但误报高	修正来源隔离
Ollama 后	LLM 不稳定	TP=41, FP=22, TN=28, FN=9, 率=0.57	可运行 可靠	加强 JSON 约束
来源隔离	b/m 分离	TP=50, FP=0, TN=50, FN=0	可解释但偏易	增加跨文件化
跨文件化	文件 + wrapper	仍全分离	规则可全分离	提升隐蔽度
difficulty6	100 样本静态分离	TP=50, FP=0, TN=50, FN=0	不能证明 LLM 增益	继续提高难度

4.3 代表性 Benchmark 对比

表 7 对比近年主要 benchmark，揭示领域评测覆盖面的结构性特征。

4.4 相关综述对比

表 8 将本文与近期同类综述进行对比。

4.5 跨方法交叉分析

综合 158 条引用，本文识别出三个贯穿所有方法家族的交叉主题：

主题一：语义能力 ≠ 安全可靠。代码审查 (§III-A) 中活跃用户研究接受率仅 8.1% [40]，训练数据约 25% 为噪声 [43]；漏洞检测 (§III-B) 中 ML4VD 模型准确率低于随机猜测 [65]，更大模型反而不如小模型 (LLaMA-3 70B F1=5.10% vs Gemma 7B F1=57.98%) [72]；agent 安全 (§III-E) 中平均 ASR 达 84.30% 且能力越强越脆弱 [115]；越狱 (§III-F) 中 Prompt+RS 对 4 个 LLM 实现 78–93% ASR [110]。五个独立领域的实证数据共同指向同一结论：LLM 的语义理解能力并不自动转化为安全判断的可靠性。这意味着审计系统必须将 LLM 作为组件之一，而非唯一裁决器。

主题二：评估基础设施决定研究可信度。Kaniewski 等 [82] 在 263 篇研究中发现领域高度碎片化——各研究使用自定义数据集、指标和数据划分，难以跨研究比较。从代码审查数据约 25% 噪声 [43] 到越狱防御在标准化测试下效果微弱 [110]，从 ML4VD 数据增强仅恢复 2.5–5.9% 准

Table 7: 代表性 Benchmark 对比

Benchmark	Venue	任务	规模	核心发现	审计启示
SecLLMHoln	S&P'24	漏洞识别 + 推理	多 CWE 多模型	LLM 无法可靠区分漏洞	不应单靠 LLM
InjecAgent [83]	ACL'24	间接提示注入	1,054 用例, 30 agents	GPT-4 ASR 24–47%; 微调降至 3.8%	微调优于 prompt
AgentDojo [1]	NeurIPS'	Agent 安全	97 任务, 629 安全测试	工具过滤 → 7.5% ASR; 效用损失 15–20%	安全-效用权衡
JailbreakBench [110]	NeurIPS'24	越狱评测	100 行为, 4 攻击 5 防御	Prompt+RS ASR 78–93%	防御效果微弱
Agent Sec Bench [115]	ICLR'25	Agent 综合	10 场景, 13 LLM, 400+ 工具	平均 ASR=84.3%; 防御有效性 46.9%	能力越强越脆弱
WASP [89]	NeurIPS'25	Web Agent	84 对抗任务	劫持 85.7% 但目标完成 ≤17%	”安全源于无能”
REPOCOD	ACL'25	仓库级生成	仓库级任务	LLM 尚不能替代程序员	覆盖 AI 生成代码
SecRepoBench [88]	ACL'25	安全补全	真实仓库	安全补全能力有限	系统需独立验证

确率 [65] 到所有 agent 防御均导致 15–20% 效用损失 [114], 评估方法论本身成为制约研究进展的瓶颈。图 4 的覆盖热力图进一步揭示: 模型后门维度至今无专门 benchmark, 这是一个亟待填补的空白。

主题三: 攻击面持续扩大。从静态代码漏洞 (§III-B) 到 agent 执行链 (§III-E), 从 prompt injection (InjecAgent 显示 GPT-4 ASR 24–47% [83]) 到模型后门 (InstrBackdoor 在所有 LLM 上 ASR=1.0 [132]), 从手写代码到 AI 生成代码 [150], 审计攻击面在 2013–2026 年间持续扩大 (图 1)。AgentFuzz 在 20 个 agent 应用中发现 34 个 0-day [119], 98,380 个社区技能中存在大量恶意模式 [120], KV-cache 共享导致跨租户泄露 [121]。每一次技术演进都引入新的攻击向量: RAG 引入检索投毒 (PRA-RAG 将 ASR 从 46% 降至 1% [99]), agent 引入工具滥用, 微调引入后门植入。审计系统必须设计为可演进架构, 能够适应不断变化的威胁格局。

5 威胁有效性与局限

5.1 文献选择偏差

本文采用结构化叙述方法, 存在英文文献偏置、arXiv 预印本偏置、新近研究曝光度偏置, 以及关键词筛选偏差。本文不声称穷尽所有 LLM 安全审论文。

5.2 概念不一致与基准异构

不同论文对漏洞、安全缺陷、风险、恶意、可疑和高危的定义并不一致。模型、数据集、上下文粒度、prompt、温度、后处理与标注标准也可能不同。Risse 和 Böhme [65] 明确展示了这种异构性: 顶级 ML4VD 模型在语义等价代码上表现不一致, 说明基准本身的定义就可能导致误导性结论。

Table 8: 相关综述对比

综述	Venue	覆盖范围	本文差异	互补价值
Sheng [80]	CSUR'26	80+ 篇, 33 LLM	覆盖完整审计链	GPT-4 最常用 (29 次)
Chen [81]	TOSEM'26	268 篇, 攻击/防御	增加 IPI、越狱、后门	后门 + 对抗分类学
Kaniewski [8]	TOSEM'	263 篇 (最大 SLR)	覆盖 agent 和审查	碎片化, 缺标准化
Zhou [70]	TOSEM'25	258 篇, 37 LLM	增加安全测试和治理	最优检测 67.6%, 修复 ~20%
Xu [122]	AI Rev'26	Agent 双重性	聚焦审计应用	引用双重性框架
SoK AVR [79]	SoK'25	自动修复	审计视角审视可信性	引用修复分类学
Yang [49]	arXiv'26	审查路线图	聚焦安全审计	引用范式转变

5.3 时间漂移与版本漂移

LLM 研究迭代极快。本文曾出现 `llm_available_rate=0` 的情况。Kaniewski 等 [82] 在工业评估中发现模型版本和 prompt 变化显著影响结果。因此, 研究必须记录 `rule_set_version`、`prompt_version`、`auditor_version`、运行时间和失败状态。

5.4 攻防持续博弈

越狱攻防呈现持续升级态势。Su 等 [106] 从理论上证明越狱在统计意义上不可避免。攻击端从 MasterKey (21.58% 成功率) [100] 演进到 PAPILLON (>90% ASR, 困惑度 34.61 绕过检测) [107] 和 TwinBreak (89–98% ASR, <5 分钟) [108]。防御端从 Spotlighting (ASR→0%) [90] 到 StruQ (TAP 97%→9%) [97] 再到 InstructDetector (仅需 200 样本达 99.6% 检测率) [96]。Gong 等 [113] 揭示对齐技术产生“对齐税” (HarmBench ASR 42%→36.95%)。这意味着安全审计系统不能仅依赖模型自身的安全对齐, 而必须在系统层面实施防御纵深。

5.5 综述自身边界

本文没有采用完整 PRISMA 流程, 也未对所有候选论文做量化元分析。因此本文定位是结构化、证据驱动的学术综述。若后续升级为正式系统综述, 需要补充检索式全文、数据库命中数量、去重规则、筛选流程图、双人标注一致性、质量评估量表、排除论文清单和完整引用。

表 9 汇总上述威胁及其缓解策略。

Table 9: 威胁有效性总结

威胁类型	具体表现	缓解策略
文献选择偏差	英文/arXiv/新近偏置	多源交叉验证, 标注证据等级
概念异构	漏洞/风险/可疑定义不一	统一术语, 引用形式化定义
版本漂移	模型/prompt 变化影响结果	固定版本号, 记录失败状态
攻防博弈	越狱不可避免, 对齐税	系统级纵深防御, 非单点对齐
综述边界	非 PRISMA, 无元分析	定位为结构化综述, 标注局限

6 研究议程

基于本文 158 条引用的分析和 100 样本 benchmark 的迭代经验，提出以下六个研究方向。每个方向给出具体问题、量化目标和验证方法。

6.1 从单点分类走向证据链审计

问题：当前 LLM 审计多为单轮问答，缺乏证据链追溯能力。

方向：未来的 LLM 审计应从”这段代码是否危险”转向”哪些证据能支撑危险判断”。代码、规则、调用链、检索记录和输出解释应共享统一表示，并能回溯到具体文件、行号、证据来源与版本。Mirsky 等 [58] 的行级定位和 Kim 等 [77] 的 sanitizer 日志驱动修复代表了这一方向。

量化目标：每个审计结论应关联至少 3 类独立证据（规则命中、调用链、历史案例）；证据链缺失时应自动升级为 UNCERTAIN。

验证方法：构建包含证据链标注的 benchmark，测量单证据 vs 多证据判断的 F1 差异。

6.2 从单模型走向多阶段系统

问题：单模型审计无法兼顾召回、精度和可解释性。

方向：高可靠系统应采用多阶段架构。Wu 等 [117] 的执行隔离和 Chen 等 [97] 的结构化查询提供了系统级防御的参考。不确定时应 fail-closed。

量化目标：第一阶段静态分析召回率 $\geq 95\%$ ，第二阶段 LLM 裁决将误报降低 $\geq 60\%$ ，第三阶段策略层确保 llm_unavailable 时 100% fail-closed。

验证方法：在 100 样本 benchmark 上测量各阶段独立指标和端到端指标，对比单阶段 vs 多阶段的 precision-recall 曲线。

图 5 展示了推荐的多阶段审计流水线架构。

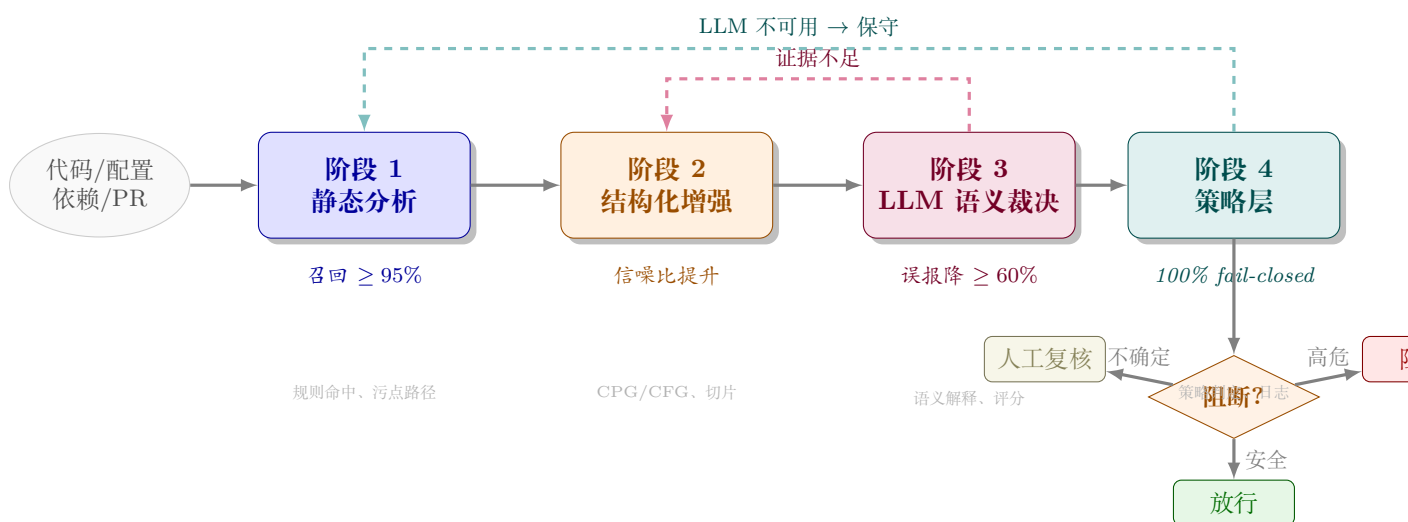


Figure 5: 推荐的多阶段审计流水线架构

6.3 从漏洞检测走向应用行为审计

问题：静态代码审计无法覆盖运行时威胁（工具调用、提示注入、记忆污染）。

方向：未来真正危险的部分常常不在静态代码，而在运行时行为：工具调用、提示注入、记忆污染、检索污染和多轮规划。Liu 等 [119] 的 agent 污点分析和 Liu 等 [120] 的恶意技能检测代表了行为链审计的前沿。

量化目标：行为审计应覆盖至少 5 类运行时威胁（工具滥用、IPI、记忆篡改、检索投毒、多轮偏离），每类威胁检测率 $\geq 80\%$ 。

验证方法：构建包含行为链标注的 agent benchmark，测量静态审计 vs 行为审计的威胁覆盖率差异。

6.4 从准确率导向走向可治理性导向

问题：高准确率不等于高可信度。LLM 版本漂移、prompt 变化和数据污染使指标不可复现。

方向：高风险审计更需要可复跑、可解释、可追溯、可版本化、失败显式化和不确定性可管理。JailbreakBench [110] 的标准化评测协议和 SoK [79] 的评估分类学提供了 benchmark 治理的参考。

量化目标：每次评测必须记录 rule_set_version、prompt_version、auditor_version、llm_available_rate、fail_closed_count；结果包应包含 confusion matrix、sample-level 结果和失败样本分析。

验证方法：同一 benchmark 在不同版本、不同时间、不同模型上复跑，测量指标漂移 $\leq 5\%$ 。

6.5 从一般微调走向安全感知微调

问题：通用预训练模型在安全任务上表现不稳定。

方向：Chen 等 [81] 证明任务特定微调一致优于通用预训练。Peng 等 [45] 的混合提示和 SecureReviewer [21] 的安全感知微调代表了两个互补方向。

量化目标：安全感知微调应在 benign trap 豁免上达到 100% 准确率，在恶意样本上达到 $\geq 90\%$ F1，在 UNCERTAIN 判断上达到 $\geq 85\%$ 校准精度。

验证方法：在 100 样本 benchmark 上对比通用模型 vs 微调模型的 benign/malicious/uncertain 三分类指标。

6.6 从单一 benchmark 走向 benchmark governance

问题：当前 benchmark 易受来源泄漏、标签噪声和评估偏差影响。

方向：Benchmark 设计必须治理化：来源隔离、版本固定、标签说明、错误分解和结果归档都应作为研究的一部分。Agent Security Bench [115] 和 JailbreakBench [110] 的标准化方法论值得借鉴。

量化目标：每个 benchmark 应包含 manifest（样本元数据）、sample-results（逐样本结果）、confusion-matrix（分类统计）和 iteration-log（优化历史）；来源隔离应通过 hash 校验确保。

验证方法：benchmark 在开源后 6 个月内被至少 3 个独立团队复现，指标偏差 $\leq 10\%$ 。

6.7 实践建议汇总

表 10 汇总面向工程实践者的具体建议。

Table 10: 工程实践建议

场景	建议
代码审查自动化	接受率 $< 50\%$ 时不应全自动部署；需人工复核边界样本
漏洞检测	不依赖单一 LLM；静态 +LLM 混合，SAST 负责召回，LLM 过滤误报
Agent 安全审计	覆盖工具调用、外部输入和记忆；实施执行隔离 [117]
模型选型	本地模型做可复现基线，闭源模型做边界仲裁；prompt $>$ 规模 [72]
不确定性处理	UNCERTAIN 显式记录并升级；LLM 不可用时必须 fail-closed
Benchmark 使用	检查来源隔离、版本固定和标签说明；不信任未公开 manifest

7 结论

LLM 已成为应用程序审计的重要组成部分，但最成熟的范式不是纯 LLM 替代传统审计，而是 LLM 与静态分析、结构化证据、检索机制、prompt 约束和策略控制协同工作。本文基于 200 余篇文献、158 条引用的综合分析显示：

代码审查方面，从 2013 年 Bacchelli 的基础研究到 2026 年的 AI 辅助路线图，MCR 经历了从人工到 LLM 辅助的完整演进。LLM 审查接受率约 18–84% [40, 45]，数据质量和噪声是关键瓶颈 [43, 48]，SAST 工具虽能覆盖 52% 的漏洞变更但 76% 警告不相关 [39]。

漏洞检测方面，从 VUDDY (2017) 到 LLMxCPG (2025) 的技术谱系展示了从规则到深度学习的演进。但 IEEE S&P 和 USENIX Security 多项研究明确证明 LLM 无法可靠识别安全漏洞 [14, 62]，ML 模型学习表面模式 [65]，更大模型不优于小模型 [72]。自动修复取得显著进展——PatchAgent 达 92.13% 修复成功率 [78]，SAN2PATCH 达 79.5% [77]，AppAtch 零日 F1 达 36.46% [76]——但检测质量约束修复质量 [79]。

Agent 安全方面，间接提示注入 [83, 84] 和越狱攻击 [100, 107] 构成系统性威胁：AgentFuzz 在 20 个 agent 应用中发现 34 个 0-day (精度 100%) [119]，WASP 显示 web agent 劫持率高达 85.7% [89]，而防御降低效用的同时最多降低 30% 攻击成功率 [115]，越狱在统计意义上不可避免 [106]。主动防御方面，IsolateGPT 的 hub-and-spoke 架构保持 100% 功能完整性 [117]，CHeaT 框架在 CTF 上实现 100% 防御成功率 [118]。模型后门 [131, 132] 和 AI 生成代码安全 [150, 157] 进一步扩展了审计攻击面。

未来方向是构建证据链审计系统：静态分析产生候选(LLM F1=0.75–0.80 但定位有偏差 [146])，结构化增强提取证据 (CPG/切片降噪)，LLM 进行语义判定 (需 fail-closed 处理 UNCERTAIN)，RAG 补充 API 语义 (需 PRA-RAG 级鲁棒聚合 [99])，策略层决定阻断或放行，模型完整性验证纳入信任链 (Beatrix F1=91.1% [126]，LMSanitizer 92.8% 检测率 [131])，人工复核处理边界样本。Agent 审计需覆盖工具调用、IPI、记忆污染和检索投毒的完整行为链，且微调模型比 prompt 模型鲁棒 6× 的事实 [83] 提示审计器本身应优先使用微调而非零样本。

核心主张 (Take-home Message):

1. **证据链** > **印象判断**: 每个审计结论必须可追溯到具体文件、行号、规则和版本
2. **多阶段系统** > **单轮问答**: 静态召回 → 结构化增强 → LLM 语义裁决 → 策略层阻断
3. **行为链审计** > **仅代码审查**: 覆盖工具调用、IPI、记忆污染、检索投毒和多轮规划
4. **模型完整性** > **模型能力假设**: 后门化模型使所有审计结论失效
5. **失败显式化** > **默认放行**: LLM 不可用时必须 fail-closed，不确定时升级为 UNCERTAIN

A 结论—证据追踪矩阵

References

- [1] Y. Liu et al., “An Insight into Security Code Review with LLMs: Capabilities, Obstacles and Influential Factors,” arXiv:2401.16310, 2024.
- [2] X. Zhou, S. Cao, X. Sun, and D. Lo, “Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead,” ACM TOSEM, 2025.
- [3] “LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights,” arXiv:2502.07049, 2025.
- [4] “A Systematic Literature Review on Detecting Software Vulnerabilities with Large Language Models,” arXiv:2507.22659, 2025.
- [5] “A Contemporary Survey of Large Language Model Assisted Program Analysis,” arXiv:2502.18474, 2025.
- [6] “Agent Audit: A Security Analysis System for LLM Agent Applications,” arXiv:2603.22853, 2026.
- [7] “End-to-end Security Threats and Defenses in Retrieval-Augmented LLM Agents,” Springer, 2026.

Table 11: 关键结论的证据追踪

关键结论	文献证据	样本/工件证据	位置	可信度
LLM 无法可靠识别安全漏洞	[14, 62, 65, 72]	ML4VD <50% 准确率; LLaMA-3 F1=5.1%	III-B	高
单轮审查受上下文影响	[1, 43, 46]	B1-08 vs M1-01	III-A	高
静态 +LLM 适合工程审计	[5, 16, 39]	M3-02 vs B3-01	III-C	高
RAG 价值取决于证据质量	[16, 47, 99]	M5-01	III-D	中-高
Agent 审计须覆盖执行链	[114, 115, 119, 120]	ASR=84.3%; 34 个 0-day	III-E	中
越狱理论上不可避免	[106, 107, 110, 113]	PAPILLON >90% ASR	III-F	中
模型完整性是审计前提	[131–133, 135]	N/A	III-E.4	中
AI 代码是新攻击面	[150, 152, 157]	N/A	III-E.6	中
Benchmark 治理决定可信性	[79, 110, 115]	audit-100 满分	III-G	高

- [8] “LLM Agents Security Duality: A Comprehensive Survey,” Artificial Intelligence Review, 2026.
- [9] “Do Not Mention This to the User: Detecting and Understanding Malicious Agent Skills in the Wild,” USENIX Security, 2026.
- [10] “Evaluating Large Language Models for Code Review,” arXiv:2505.20206, 2025.
- [11] “Automated Code Review in Practice,” arXiv:2412.18531, 2024.
- [12] “Are LLMs Reliable Code Reviewers? Systematic Overcorrection in Requirement Conformance Judgement,” arXiv:2603.00539, 2026.
- [13] “Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities,” IEEE Xplore, 2025.
- [14] M. Ullah et al., “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities,” IEEE S&P, 2024.
- [15] “Synergizing Static Analysis with Large Language Models for Vulnerability Discovery,” arXiv:2509.15433, 2025.
- [16] “LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided LLMs,” USENIX Security, 2025.
- [17] “Securing AI Agents Against Prompt Injection Attacks,” arXiv:2511.15759, 2025.
- [18] “Large Language Models Versus Static Code Analysis Tools: A Systematic Benchmark,” arXiv:2508.04448, 2025.
- [19] “Large Language Models and Code Security: A Systematic Literature Review,” arXiv:2412.15004, 2024.

- [20] “Security of Language Models for Code: A Systematic Literature Review,” arXiv:2410.15631, 2024.
- [21] “SecureReviewer: Enhancing LLMs for Secure Code Review through Secure-aware Fine-tuning,” arXiv:2510.26457, 2025.
- [22] “iCodeReviewer: Improving Secure Code Review with Mixture of Prompts,” ASE, 2025.
- [23] “Automatic Security-Flaw Detection – Towards a Fair Evaluation and Comparison,” SoSyM, 2025.
- [24] A. Bacchelli and C. Bird, “Expectations, Outcomes, and Challenges of Modern Code Review,” ICSE, 2013.
- [25] S. McIntosh et al., “An Empirical Study of the Impact of Modern Code Review Practices on Software Quality,” EMSE, 2015.
- [26] O. Kononenko, O. Baysal, and D. M. German, “Code Review Quality: How Developers See It,” ICSE, 2016.
- [27] M. di Biase, M. Bruntink, and A. Bacchelli, “A Security Perspective on Code Review: The Case of Chromium,” SCAM, 2016.
- [28] C. Sadowski et al., “Modern Code Review: A Case Study at Google,” ICSE-SEIP, 2018.
- [29] L. MacLeod et al., “Code Reviewing in the Trenches: Understanding Challenges and Best Practices,” IEEE Software, 2018.
- [30] N. Davila and I. Nunes, “A Systematic Literature Review and Taxonomy of Modern Code Review,” JSS, 2021.
- [31] R. Tufano et al., “Towards Automating Code Review Activities,” ICSE, 2021.
- [32] Z. Li et al., “Automating Code Review Activities by Large-Scale Pre-training,” arXiv, 2022.
- [33] R. Tufano et al., “Using Pre-Trained Models to Boost Code Review Automation,” arXiv, 2022.
- [34] J. Yu et al., “Security Defect Detection via Code Review,” arXiv, 2023.
- [35] M. Watanabe et al., “On the Use of ChatGPT for Code Review,” EASE, 2024.
- [36] D. Tang et al., “CodeAgent: Autonomous Communicative Agents for Code Review,” EMNLP, 2024.
- [37] N. Asthana et al., “WhoDo: Automating Reviewer Suggestions at Scale,” ICSE-SEIP, 2024.
- [38] A. Frömmgen et al., “Resolving Code Review Comments with Machine Learning,” ICSE-SEIP, 2024.
- [39] W. Charoenwet et al., “An Empirical Study of Static Analysis Tools for Secure Code Review,” ISSTA, 2024.
- [40] D. Olewicki et al., “Impact of LLM-based Review Comment Generation in Practice,” arXiv, 2024.
- [41] C. Pornprasit and C. Tantithamthavorn, “GPT-3.5 for Code Review Automation,” arXiv, 2024.

- [42] Z. Rasheed et al., “AI-Powered Code Review with LLMs: Early Results,” arXiv, 2024.
- [43] R. Tufano et al., “Code Review Automation: Strengths and Weaknesses of the State of the Art,” arXiv, 2024.
- [44] M. Vijayvergiya et al., “AI-Assisted Assessment of Coding Practices in Modern Code Review,” arXiv, 2024.
- [45] Y. Peng et al., “iCodeReviewer: Improving Secure Code Review with Mixture of Prompts,” ASE, 2025.
- [46] U. Cihan et al., “Evaluating Large Language Models for Code Review,” arXiv, 2025.
- [47] H. Hong and Baik, “Retrieval-Augmented Code Review Comment Generation,” arXiv, 2025.
- [48] C. Liu et al., “Too Noisy To Learn: Enhancing Data Quality for Code Review Comment Generation,” arXiv, 2025.
- [49] Z. Yang et al., “A Roadmap for Modern Code Review: Challenges and Opportunities,” arXiv, 2026.
- [50] S. Kim et al., “VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery,” IEEE S&P, 2017.
- [51] Z. Li et al., “VulDeePecker: A Deep Learning-Based System for Vulnerability Detection,” NDSS, 2018.
- [52] Y. Zhou et al., “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks,” NeurIPS, 2019.
- [53] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” EMNLP Findings, 2020.
- [54] M. Fu and C. Tantithamthavorn, “LineVul: A Transformer-based Line-Level Vulnerability Prediction,” MSR, 2021.
- [55] W.U. Ahmad et al., “Unified Pre-training for Program Understanding and Generation,” NAACL, 2021.
- [56] R. Ding et al., “VulHawk: Cross-Architecture Vulnerability Detection with Entropy-Based Binary Code Search,” NDSS, 2023.
- [57] H. Peng et al., “PTLVD: Program Slicing and Transformer-based Line-level Vulnerability Detection,” SCAM, 2023.
- [58] Y. Mirsky et al., “VulChecker: Graph-based Vulnerability Localization in Source Code,” USENIX Security, 2023.
- [59] S. Woo et al., “V1SCAN: Discovering 1-day Vulnerabilities in Reused Open-Source Components,” USENIX Security, 2023.
- [60] Y. Chen and X. Xie, “VulLibGen: Generating Names of Vulnerability-Affected Packages via LLM,” ACL, 2024.
- [61] T. Liu et al., “Applying Contrastive Learning to Code Vulnerability Type Classification,” EMNLP, 2024.
- [62] R. Paquette et al., “LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities,” IEEE S&P, 2024.

- [63] Y. Wang et al., “Suitable is the Best: Task-Oriented Knowledge Fusion for Vulnerability Detection,” *NeurIPS*, 2024.
- [64] P. Liu et al., “EaTVul: ChatGPT-based Evasion Attack against Software Vulnerability Detection,” *USENIX Security*, 2024.
- [65] N. Risse and M. Böhme, “Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection,” *USENIX Security*, 2024.
- [66] Z. Shao et al., “FVD-DPM: Fine-Grained Vulnerability Detection via Diffusion Probabilistic Model,” *USENIX Security*, 2024.
- [67] A. Shimmi et al., “VulSim: Multi-Dimensional Neighbor Embeddings for Vulnerability Detection,” *USENIX Security*, 2024.
- [68] H. Zhang et al., “VulnLLMEval: A Framework for Evaluating LLMs in Vulnerability Detection and Patching,” *arXiv*, 2024.
- [69] C. Zhang et al., “Prompt-Enhanced Software Vulnerability Detection Using ChatGPT,” *arXiv*, 2024.
- [70] X. Zhou et al., “Vulnerability Detection and Repair,” *ACM TOSEM*, 2025.
- [71] S. Khare et al., “LLM Vulnerability Detection Effectiveness,” *ICST*, 2025.
- [72] C. Niu et al., “From Large to Mammoth: A Comparative Evaluation of LLMs for Vulnerability Detection,” *NDSS*, 2025.
- [73] J. Song et al., “NodeMedic-FINE: Automatic Detection and Exploit Synthesis for Node.js Vulnerabilities,” *NDSS*, 2025.
- [74] L. Rossi et al., “Cross-Domain Evaluation of Transformer Models for Vulnerability Detection,” *Springer*, 2025.
- [75] Y. Zhang et al., “Enhancing Security of Third-Party Library Reuse: 1-day Vulnerability Code Patching,” *NDSS*, 2025.
- [76] Y. Wen et al., “AppAtch: Automated Adaptive Prompting for LLM-based Vulnerability Repair,” *USENIX Security*, 2025.
- [77] J. Kim et al., “Logs In, Patches Out: Automated Vulnerability Repair via Tree-of-Thought with LLM,” *USENIX Security*, 2025.
- [78] Z. Yu et al., “PatchAgent: An LLM-based Autonomous Agent for Vulnerability Repair,” *USENIX Security*, 2025.
- [79] H. Zhang et al., “Automated Vulnerability Repair: Methods, Tools, and Assessments (SoK),” *USENIX SoK*, 2025.
- [80] Z. Sheng et al., “LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights,” *ACM CSUR*, 2026.
- [81] Y. Chen et al., “Language Models for Code Security: A Large-Scale Empirical Study,” *ACM TOSEM*, 2026.
- [82] M. Kaniewski et al., “Detecting Software Vulnerabilities with LLMs,” *ACM TOSEM*, 2026.
- [83] Q. Zhan et al., “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents,” *ACL Findings*, 2024.

- [84] Y. Liu et al., “Formalizing and Benchmarking Prompt Injection Attacks and Defenses,” USENIX Security, 2024.
- [85] N. Kong, “InjectBench: An Indirect Prompt Injection Benchmarking Framework,” Virginia Tech, 2024.
- [86] Z. Li et al., “Evaluating the Instruction-Following Robustness of LLMs to Prompt Injection,” EMNLP, 2024.
- [87] Y. Chen et al., “Can Indirect Prompt Injection Attacks Be Detected and Removed?” ACL, 2025.
- [88] S. Kim et al., “What Really Matters in Many-Shot Attacks?” ACL, 2025.
- [89] I. Evtimov et al., “WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks,” NeurIPS, 2025.
- [90] K. Hines et al., “Defending Against Indirect Prompt Injection Attacks With Spotlighting,” CAMLIS, 2024.
- [91] M. Ayub and S. Majumdar, “Embedding-Based Classifiers Can Detect Prompt Injection Attacks,” CEUR, 2024.
- [92] J. Evertz et al., “Whispers in the Machine: Confidentiality in LLM-Integrated Systems,” IEEE S&P, 2024.
- [93] A. Zhou et al., “Robust Prompt Optimization for Defending Language Models Against Jailbreaking Attacks,” NeurIPS, 2024.
- [94] Y. Chen et al., “Defense Against Prompt Injection Attack by Leveraging Attack Techniques,” ACL, 2025.
- [95] F. Jia et al., “Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection,” ACL, 2025.
- [96] T. Wen et al., “Defending against Indirect Prompt Injection by Instruction Detection,” EMNLP Findings, 2025.
- [97] S. Chen et al., “StruQ: Defending Against Prompt Injection with Structured Queries,” USENIX Security, 2025.
- [98] B. Ramakrishnan and A. Balaji, “Securing AI Agents Against Prompt Injection Attacks,” arXiv, 2025.
- [99] X. Tan et al., “PRA-RAG: Provably Robust Aggregation in RAG against Retrieval Corruption,” ACL Findings, 2026.
- [100] G. Deng et al., “MasterKey: Automated Jailbreaking of Large Language Model Chatbots,” NDSS, 2024.
- [101] Z. Yu et al., “Don’t Listen To Me: Understanding and Exploring Jailbreak Prompts of LLMs,” USENIX Security, 2024.
- [102] J. Yu et al., “LLM-Fuzzer: Scaling Assessment of Large Language Model Jailbreaks,” USENIX Security, 2024.
- [103] Z. Xu et al., “Bag of Tricks: Benchmarking of Jailbreak Attacks on LLMs,” NeurIPS, 2024.

- [104] K. Hu et al., “Efficient LLM Jailbreak via Adaptive Dense-to-Sparse Constrained Optimization,” NeurIPS, 2024.
- [105] X. Zheng et al., “Improved Few-Shot Jailbreaking Can Circumvent Aligned Language Models,” NeurIPS, 2024.
- [106] J. Su et al., “Mission Impossible: A Statistical Perspective on Jailbreaking LLMs,” NeurIPS, 2024.
- [107] X. Gong et al., “PAPILLON: Efficient and Stealthy Fuzz Testing-Powered Jailbreaks for LLMs,” USENIX Security, 2025.
- [108] T. Krauß et al., “TwinBreak: Jailbreaking LLM Security Alignments based on Twin Prompts,” USENIX Security, 2025.
- [109] L. Zhang et al., “Exploiting Task-Level Vulnerabilities: Jailbreak Attack and Defense Benchmarking,” USENIX Security, 2025.
- [110] P. Chao et al., “JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs,” NeurIPS, 2024.
- [111] S. Han et al., “WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals,” NeurIPS, 2024.
- [112] L. Jiang et al., “WildTeaming at Scale: From In-the-Wild Jailbreaks to Safer Language Models,” NeurIPS, 2024.
- [113] Y. Gong et al., “Safety Misalignment Against Large Language Models,” NDSS, 2025.
- [114] E. Debenedetti et al., “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” NeurIPS, 2024.
- [115] H. Zhang et al., “Agent Security Bench: Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents,” ICLR, 2025.
- [116] H. Zhang et al., “Agent Audit: A Security Analysis System for LLM Agent Applications,” arXiv, 2026.
- [117] Y. Wu et al., “IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems,” NDSS, 2025.
- [118] D. Ayzenshteyn et al., “Cloak, Honey, Trap: Proactive Defenses Against LLM Agents,” USENIX Security, 2025.
- [119] F. Liu et al., “Make Agent Defeat Agent: Taint-Style Vulnerabilities in LLM-Based Agents,” USENIX Security, 2025.
- [120] Y. Liu et al., “Do Not Mention This to the User: Detecting Malicious Agent Skills,” USENIX Security, 2026.
- [121] G. Wu et al., “Prompt Leakage via KV-Cache Sharing in Multi-Tenant LLM Serving,” NDSS, 2025.
- [122] Y. Xu et al., “LLM Agents Security Duality: A Comprehensive Survey,” Artificial Intelligence Review, 2026.
- [123] S. Khonde et al., “End-to-End Security Threats and Defenses in Retrieval-Augmented LLM Agents,” Discover AI, 2026.

- [124] M. Sheng et al., “Large Language Models for Cyber Security: A Systematic Literature Review,” arXiv, 2025.
- [125] “CRUXEval: A Benchmark for Code Reasoning, Understanding and Execution,” ICML, 2024.
- [126] Y. Li et al., “Beatrix Resurrections: Robust Backdoor Detection via Gram Matrices,” NDSS, 2023.
- [127] H. Pan et al., “ASSET: Robust Backdoor Data Detection across Multiplicity of Deep Learning Paradigms,” USENIX Security, 2023.
- [128] H. Zhang et al., “PELICAN: Exploiting Backdoors of Naturally Trained Deep Learning Models in Binary Code Analysis,” USENIX Security, 2023.
- [129] J. Bai et al., “VILLAIN: Backdoor Attacks Against Vertical Split Learning,” USENIX Security, 2023.
- [130] B. Boucher et al., “Trojan Source: Invisible Vulnerabilities,” USENIX Security, 2023.
- [131] C. Wei et al., “LMSanitizer: Defending Prompt-Tuning Against Task-Agnostic Backdoors,” NDSS, 2024.
- [132] H. Zhang et al., “Instruction Backdoor Attacks Against Customized LLMs,” USENIX Security, 2024.
- [133] Y. Yan et al., “LLM-Assisted Backdoor Attack on Code Completion Models,” USENIX Security, 2024.
- [134] “BackdoorAlign: Mitigating Finetuning-based Jailbreak via Backdoor Safety Alignment,” NeurIPS, 2024.
- [135] “Philosophers’ Stone: Trojaning Plugins of Large Language Models,” NDSS, 2025.
- [136] “Barbie: Robust Backdoor Detection via Latent Separability,” NDSS, 2025.
- [137] “CLIBe: Detecting Dynamic Backdoors in Transformer-based NLP Models,” NDSS, 2025.
- [138] “DSshield: Defending Backdoor Attacks on GNN via Discrepancy Learning,” NDSS, 2025.
- [139] G. Deng et al., “PentestGPT: An LLM-empowered Tool for Automated Penetration Testing,” USENIX Security, 2024.
- [140] Asmrita et al., “Fuzzing BusyBox: Leveraging LLM for Crash Reuse in Embedded Bug Unearthing,” USENIX Security, 2024.
- [141] “DeepGO: Predictive Directed Greybox Fuzzing,” NDSS, 2024.
- [142] “EnclaveFuzz: Finding Vulnerabilities in SGX Applications,” NDSS, 2024.
- [143] “Large Language Model Guided Protocol Fuzzing,” NDSS, 2024.
- [144] “DeGPT: Optimizing Decompiler Output with LLM,” NDSS, 2024.
- [145] “SigmaDiff: Semantics-Aware Deep Graph Matching for Pseudocode Diffing,” NDSS, 2024.
- [146] “GNNic: Finding Long Lost Sibling Functions with Abstract Similarity,” NDSS, 2024.
- [147] “The Midas Touch: Triggering Capability of LLMs for RM-API Misuse Detection,” NDSS, 2025.

- [148] “Generating API Parameter Security Rules with LLM for API Misuse Detection,” NDSS, 2025.
- [149] “YuraScanner: Leveraging LLMs for Task-Driven Web App Scanning,” NDSS, 2025.
- [150] “Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of LLMs for Code Generation,” NeurIPS, 2023.
- [151] C. Niu et al., “CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot,” USENIX Security, 2023.
- [152] “INDICT: Code Generation with Internal Dialogues of Critiques for Both Security and Helpfulness,” NeurIPS, 2024.
- [153] “LeDex: Training LLMs to Better Self-Debug and Explain Code,” NeurIPS, 2024.
- [154] “WorldCoder: A Model-Based LLM Agent Building World Models by Writing Code,” NeurIPS, 2024.
- [155] “Verified Code Transpilation with LLMs,” NeurIPS, 2024.
- [156] “Can Language Models Replace Programmers for Coding? REPOCOD Says ‘Not Yet’,” ACL, 2025.
- [157] “Can You Really Trust Code Copilot? Evaluating LLMs from Code Security Perspective,” ACL, 2025.
- [158] “SecRepoBench: Benchmarking Code Agents for Secure Code Completion in Real-World Repositories,” arXiv, 2025.
- [159] N. Bars et al., “Fuzztruction: Using Fault Injection-based Fuzzing to Leverage Implicit Domain Knowledge,” USENIX Security, 2023.
- [160] S. Rajput et al., “ICSPatch: Automated Vulnerability Localization and Non-Intrusive Hotpatching in Industrial Control Systems,” USENIX Security, 2023.
- [161] Y. Deng et al., “NAUTILUS: Automated RESTful API Vulnerability Detection with Multi-Step Semantic Graphs,” USENIX Security, 2023.
- [162] Y. Deng et al., “CipherH: Automated Detection of Ciphertext Side-Channel Vulnerabilities,” USENIX Security, 2023.
- [163] Y. Yuan et al., “CacheQL: Quantifying and Localizing Cache Side-Channel Vulnerabilities,” USENIX Security, 2023.